From Decisions to Estimation: Linear Neuron and Loss

What Changed from the Last Lab

In Lab 2 and Lab 3, the output was a **decision**.

- ACCEPT / REJECT
- Class 0 / Class 1

Even when we moved from perceptron to logistic regression, the end goal was still a **decision**, with probability added only to express confidence.

In DLQ3, that assumption broke.

Here, the output is not a category.

The output is a **number**.

That single change forces everything else to change.

Why a Decision Function Fails Here

A decision function answers only this:

> “On which side of the boundary does this input lie?”

That is enough when outputs are discrete.

For salary estimation:

- Two candidates can be on the **same side** of every threshold
- Yet their salaries must be **different**

So the model must preserve **magnitude**, not just direction.

This immediately rules out:

- Step function
- Sigmoid function

Not because they are “bad”, but because they **destroy numeric scale**.

The Simplest Model That Can Output Numbers

We strip everything back.

One neuron.

No activation function.

Linear neuron (single layer, no activation)

$$\hat{y} = w_1 x_1 + w_2 x_2 + b$$

Where:

- x1: Years of Experience
- x2: Test Score
- w1,w2: importance of each input
- b: baseline salary offset
- y^: predicted salary

This is not a new architecture.

It is the **same perceptron**, with one deliberate change:

> We removed the decision function.

Why This Is Still a Perceptron

Nothing structural changed.

The model still:

- multiplies inputs by importance
- adds them
- shifts with bias

The only thing missing is the final decision step.

Earlier:

$$z = \sum w_i x_i + b$$

Then:

$$\hat{y} = \text{step}(z) \quad \text{or} \quad \sigma(z)$$

Now:

$$\hat{y} = z$$

Same computation.

Different interpretation.

What "Being Wrong" Means Now

In classification, error was:

$$error = y - \hat{y} \in \{-1, 0, 1\}$$

That made sense only because outputs were discrete.

Here, outputs are continuous.

So error must capture **distance**, not mismatch.

Per-sample error

$$e = \hat{y} - y$$

This value tells:

- sign → overestimate or underestimate
- magnitude → how bad the mistake is

But one error is not enough.

We need a **single quantity** that summarizes performance over data.

Why We Need a Loss Function

A model cannot learn from raw errors unless we define:

> "What does the model try to minimize?"

This is the role of a **loss function**.

A loss function converts errors into a **single scalar objective**.

That objective must:

- penalize large mistakes more than small ones
- be smooth (so we can adjust parameters gradually)

Mean Squared Error (Why This Loss)

Definition

$$L_{\text{MSE}} = \frac{1}{N} \sum_{i=1}^{N} (\hat{y}_i - y_i)^2$$

Why square the error

1. Squaring removes sign

   Overestimates and underestimates are both penalized.

2. Squaring amplifies large errors

   A ₹10,000 error matters more than two ₹5,000 errors.

3. Squaring makes the function smooth

   Which allows gradient-based updates.

This directly matches the scenario:

> "Small errors are acceptable. Large errors are problematic."

What Learning Can Change

Learning **cannot** change:

- input values

- output definition
- model structure

Learning **must** change:

- importance of inputs
- baseline offset

That means:

$$w_1, w_2, b \quad \text{must change}$$

Nothing else.

This is exactly the same conclusion as in Lab 2 and Lab 3.

Only the **objective** changed.

How Learning Happens

Learning is not guessing new rules.

Learning is:

> Adjusting and b so that
>
> LMSE decreases.

This requires knowing:

- how loss changes when a weight changes
- in which direction to move the weight

That is what gradients provide.

10. Gradients for Linear Regression

For the linear neuron:

$$\hat{y}_i = w_1 x_{i1} + w_2 x_{i2} + b$$

Gradients of MSE:

$$\frac{\partial L}{\partial w_j} = \frac{2}{N} \sum_{i=1}^{N} (\hat{y}_i - y_i) x_{ij}$$

$$\frac{\partial L}{\partial b} = \frac{2}{N} \sum_{i=1}^{N} (\hat{y}_i - y_i)$$

Interpretation:

- if predictions are too high, gradients are positive → weights decrease
- if predictions are too low, gradients are negative → weights increase
- inputs scale the correction automatically

This is not a new idea.

It is the same **error-driven correction** you saw earlier, now continuous.

Parameter Update Rule

With learning rate η\etaη:

$$w_j \leftarrow w_j - \eta \frac{\partial L}{\partial w_j}$$

$$b \leftarrow b - \eta \frac{\partial L}{\partial b}$$

This is gradient descent.

Nothing more.

Connecting Back to Previous Labs

- Lab 2:

  Error changed weights to fix wrong decisions.

- Lab 3:

  Cross-entropy loss shaped probability outputs.

- Lab 4:

MSE shapes numeric predictions.

Same learning principle.

Different loss.

Different output interpretation.

Where This Leads Next

Once this is clear:

- multi-feature linear regression is just more inputs
- vectorized form appears naturally
- scaling and normalization become necessary
- non-linear patterns require feature transforms or deeper networks

But all of that sits **on top of this exact model**.